CS 7180: Claude Code Extensibility

John Alexis Guerra Gomez
jguerra at northeastern.edu
Class: **johnguerra.co/classes/aiCoding_spring_2026**
Slides: **johnguerra.co/lectures/ai_assisted_coding**

What We'll Cover Today

1. Where We Are -- Week 12 checkpoint
2. Skills & Custom Commands
3. Hooks Deep Dive
4. MCP Servers
5. Custom Sub-agents
6. Parallel Development with Worktrees
7. Agent Teams (Experimental)
8. Code Review for AI-Generated Code
9. Hands-on Lab

Where We Are

> *Week 12 -- Extending Claude Code for team-scale development*

Recap: Weeks 10-11

Week 10: Claude Code foundations -- agentic loop, CLAUDE.md, tools, permissions, context management
Week 11: Workflows -- Explore-Plan-Implement-Commit, TDD, Git/GitHub integration, CI/CD, non-interactive mode
You know how Claude Code works and how to use it.
Now you learn to *extend* it for your team's specific needs.

This Week: Extensibility & Parallel Work

Today's focus: making Claude Code do what you need

- **Skills** -- reusable slash commands for your team
- **Hooks** -- deterministic automation at lifecycle points
- **MCP servers** -- connecting Claude Code to external tools
- **Sub-agents** -- specialized AI reviewers, auditors, test writers
- **Parallel development** -- multiple agents working simultaneously
- **Code review** -- reviewing AI-generated code with the C.L.E.A.R. framework

All of these directly support your **P3 Sprint 2** work.

Skills & Custom Commands

> *Reusable workflows your whole team shares*

What Are Skills?

Skills are markdown files in `.claude/skills/` that teach Claude Code **how to perform specific tasks**.

```
your-project/
  .claude/
    skills/
      fix-issue.md       <-- "/fix-issue" slash
command
      deploy.md          <-- "/deploy" slash command
      add-component.md   <-- "/add-component" slash
command
```

When you type `/fix-issue`, Claude Code loads the skill file and follows its instructions.

Anatomy of a Skill File

```
# Fix Issue
## Description
Fix a GitHub issue: branch, implement, test, PR.
## Steps
1. Read issue details from GitHub
2. Create branch fix/{issue-number}
3. Implement following project conventions
4. Write/update tests for changed code
5. Run test suite to verify
6. Create PR linking to the issue
## Constraints
- Never modify unrelated files
- Always include a test for the fix
```

Skills vs CLAUDE.md

Aspect	CLAUDE.md	Skills
Loaded	Every session, automatically	On demand, when invoked
Purpose	Project context and rules	Task-specific workflows
Scope	Universal project knowledge	One specific workflow
Triggered by	Auto-loaded	`/command` or auto-matched
Best for	Conventions, architecture	Repeatable multi-step tasks

Rule of thumb: If Claude should *always* know it, put it in CLAUDE.md. If it's a *specific workflow*, make it a skill.

Invocation: Auto vs Manual

Skills can be triggered two ways:

Auto-invoked: Claude matches your prompt to the skill description. "Create a Button component" may trigger an `add-component` skill automatically.

Manual only: Add `disable-model-invocation: true` to the skill's YAML frontmatter. This ensures the skill only runs when you explicitly type `/deploy`.

Use this for potentially destructive operations like deployment, database migrations, or cleanup tasks where you want explicit human intent.

Sharing Skills Across Teams

Skills are committed to your repo in `.claude/skills/`. This means:

- **Every team member** gets the same workflows
- **New members** ramp up instantly -- `/fix-issue` works the same for everyone
- **Skills evolve** via pull requests, just like code
- **Skills are versioned** in git history

```
git add .claude/skills/fix-issue.md
git commit -m "Add fix-issue skill for standardized
bug fix workflow"
```

Your team's best practices are encoded as repeatable, shareable commands.

Hooks Deep Dive

> *Deterministic automation at lifecycle points*

What Are Hooks?

Hooks run **shell commands** at specific points in Claude Code's lifecycle. Unlike CLAUDE.md instructions (which are advisory), hooks are **deterministic** -- they always execute.

```
Advisory (CLAUDE.md):
  "Please run prettier after editing files"
  --> Claude might forget

Deterministic (Hook):
  PostToolUse -> run prettier
  --> Always runs, every time
```

Hook Lifecycle Events

Three hook events you can tap into:

```
Think --> PreToolUse --> Execute Tool
                |
          PostToolUse
                |
    More work? --Yes--> Think
                |
              No
               v
             Stop
```

- **PreToolUse** -- before a tool (Read, Edit, Bash) runs
- **PostToolUse** -- after a tool completes
- **Stop** -- agent is about to give its final response

Configuring Hooks

Hooks live in `.claude/settings.json`:

```
{ "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "command": "npx prettier --write
$CLAUDE_FILE_PATH"
    }],
    "PreToolUse": [{
      "matcher": "Edit",
      "command": "bash .claude/hooks/check-
protected.sh"
    }]
} }
```

The `matcher` filters which tool triggers the hook. Without a matcher, the hook runs for every tool call.

Exit Codes

Hook exit codes control what happens next:

Exit Code	Meaning	Effect
0	Success	Continue normally
2	Block with feedback	Tool call is blocked; stdout is shown to Claude as feedback
Non-zero (other)	Error	Hook failure is reported, execution continues

Exit code 2 is powerful -- it lets hooks **reject actions** and explain why.

Hook Examples

Auto-format on edit (PostToolUse hook):

```
{ "matcher": "Edit|Write",
  "command": "npx prettier --write $CLAUDE_FILE_PATH"
}
```

Block edits to protected files (PreToolUse hook, exit code 2):

```
#!/bin/bash
# .claude/hooks/check-protected.sh
PROTECTED=(".env" ".env.local" "secrets.json")
for p in "${PROTECTED[@]}"; do
  if [[ "$CLAUDE_FILE_PATH" == *"$p"* ]]; then
    echo "BLOCKED: Cannot edit $p."; exit 2
  fi
done
exit 0
```

Exit code 2 blocks the edit and feeds the message back to Claude. Exit code 0 means continue.

Hooks vs CLAUDE.md

Aspect	CLAUDE.md	Hooks
Enforcement	Advisory -- Claude *should* follow	Deterministic -- always executes
Mechanism	Natural language instructions	Shell commands
Reliability	~90% (can be forgotten)	100% (guaranteed)
Best for	Style, conventions, preferences	Formatting, linting, access control
Feedback	Claude reads and interprets	Exit codes and stdout

Use CLAUDE.md for guidelines. **Use hooks** for rules that must never be broken.

MCP Servers

> *The "USB-C of AI" -- connecting Claude Code to everything*

What Is MCP?

Model Context Protocol -- a standard for connecting AI tools to external data and services.

MCP servers translate between Claude Code's tool-use protocol and external APIs. Claude Code calls MCP tools just like its built-in tools.

Why MCP Matters

Without MCP:

```
You: "What tables exist in our database?"
You: *open pgAdmin, run query, copy results*
You: *paste into Claude Code*
```

With MCP:

```
You: "What tables exist in our database?"
Claude: *calls PostgreSQL MCP server*
Claude: "Your database has 12 tables..."
```

MCP eliminates the copy-paste loop between Claude Code and external tools.

Adding an MCP Server

```
# Add a server for the current project
claude mcp add postgres -- \
  npx @anthropic/mcp-server-postgres \
  --connection-string "$DATABASE_URL"

# Add a Playwright server for browser testing
claude mcp add playwright -- \
  npx @anthropic/mcp-server-playwright

# Add a Figma server
claude mcp add figma -- \
  npx @anthropic/mcp-server-figma \
  --token "$FIGMA_TOKEN"
```

Servers run as child processes. Claude Code discovers their tools automatically.

.mcp.json for Team Sharing

Share MCP configurations via `.mcp.json` in your repo:

```
{ "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["@anthropic/mcp-server-postgres",
        "--connection-string", "$DATABASE_URL"] },
    "playwright": {
      "command": "npx",
      "args": ["@anthropic/mcp-server-playwright"] }
} }
```

Commit to git -- every teammate gets the same MCP connections automatically.

Tool Search: Scaling MCP

When you add many MCP servers, their **tool definitions consume context**.
Claude Code activates **Tool Search** automatically when tool definitions exceed ~10% of the context window.

```
Before Tool Search:
  All tool definitions loaded in context
  (hundreds of tools = thousands of tokens)

With Tool Search:
  Tools indexed separately
  Claude queries for relevant tools on demand
  Only matched tools enter context
```

You don't need to configure this -- it activates automatically as you add more MCP servers.

Building a Simple MCP Server

An MCP server exposes tools via the MCP protocol:

```
import { McpServer } from
  "@modelcontextprotocol/sdk/server";
const server = new McpServer({ name: "my-tools" });

server.tool("get_user_count",
  "Returns the number of registered users",
  {}, async () => {
    const count = await db.users.count();
    return { content: [{ type: "text",
      text: `There are ${count} users.` }] };
});
```

This course's own Canvas integration uses a custom MCP server (`tools/canvas-extras-mcp/`).

Custom Sub-agents

> *Specialized AI workers with isolated context*

What Are Sub-agents?

Sub-agents are **specialized Claude Code instances** defined in `.claude/agents/`:

```
.claude/
  agents/
    security-reviewer.md
    test-writer.md
    docs-updater.md
```

Each sub-agent has its own **instructions, constraints, and context window**. They're like specialized teammates you can call on.

Anatomy of a Sub-agent

Sub-agent files live in `.claude/agents/` with YAML frontmatter specifying `name` and `isolation`.
Example: `security-reviewer.md` with frontmatter `name: security-reviewer` and `isolation: worktree`
The body contains the role, instructions, and output format:

- **Role:** Review code for security vulnerabilities
- **Instructions:** Check for SQL injection, XSS, CSRF, input validation, hardcoded secrets, dependency CVEs
- **Output:** Checklist with severity ratings (HIGH/MED/LOW)

Sub-agents get their own context window and can operate in an isolated worktree.

isolation: worktree

The `isolation: worktree` setting gives the sub-agent its own **git worktree** -- a separate working directory.

```
Main worktree:     ~/project/          (your work)
Sub-agent worktree: ~/project/.claude/
        worktrees/security-reviewer/
                          (agent's sandbox)
```

Why isolation?

- Sub-agent edits don't interfere with your work
- Multiple sub-agents can run in parallel
- Each gets its own branch and file state
- Failed experiments are easily discarded

When to Use Sub-agents vs Skills

Feature	Skills	Sub-agents
Runs in	Your main context	Isolated context window
File access	Your working tree	Own worktree (if isolated)
Best for	Defined step-by-step workflows	Open-ended review, analysis
Context	Shares your context	Has its own context
Example	"Deploy to production"	"Review this PR for security"

Skills are recipes. **Sub-agents** are specialists.

Parallel Development with Worktrees

> *Multiple Claude Code instances working simultaneously*

The --worktree Flag

```
claude --worktree
```

Creates a **new git worktree** in `.claude/worktrees/` with its own branch. Claude Code works there without touching your main working directory.

```
~/project/
      (main worktree -- your code)
~/project/.claude/worktrees/
  feature-auth/
 (worktree 1 -- auth feature)
  feature-search/             (worktree 2 -- search
feature)
  fix-header-bug/             (worktree 3 -- bug fix)
```

All worktrees share the same git history but have independent file states.

Pattern 1: Multiple Terminals

Open several terminal windows, each running Claude Code on a different task:

```
Terminal 1: claude --worktree
  > "Implement the user profile API endpoint"

Terminal 2: claude --worktree
  > "Add search functionality to the product list"

Terminal 3: claude --worktree
  > "Fix the header layout bug on mobile"
```

Each works in its own worktree. Merge results via git when done.

Pattern 2: Competitive Solutions

Ask two agents to solve the **same problem** independently, then compare:

```
Terminal 1: claude --worktree
  > "Implement caching for the API using Redis"

Terminal 2: claude --worktree
  > "Implement caching for the API using
  >  in-memory LRU cache"
```

Review both solutions. Pick the better one. Discard the other.
Useful when you're unsure about the right approach.

Patterns 3 & 4: Background & Specialists

Background orchestration: Fire-and-forget with `run_in_background`:

```
> Run a sub-agent in the background to add
> comprehensive tests for the auth module.
> I'll keep working on the API routes.
```

Specialist sub-agents: Combine sub-agents with worktrees:

```
Main session:      "Working on the checkout flow"
Background 1:      security-reviewer (vulnerabilities)
Background 2:      test-writer (tests for new code)
Background 3:      docs-updater (API documentation)
```

Each specialist has its own context and worktree. Results merge back via git.

Best Practices for Parallel Work

1. **Scope tasks clearly** -- each agent needs a well-defined goal
2. **Avoid file conflicts** -- don't assign overlapping files to parallel agents
3. **Start with 2-3 agents** -- scale up as you build confidence
4. **Review before merging** -- parallel work needs careful integration
5. **Use Plan mode first** -- have each agent plan before implementing

For P3 teams: 3-5 teammates, each running 5-6 tasks in parallel. That's 15-30 parallel workstreams -- coordinate via your scrumboard.

Agent Teams (Experimental)

> *Multi-agent coordination with a team lead*

What Are Agent Teams?

An experimental feature where **multiple Claude Code instances** coordinate through a team lead:
`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`

The team lead breaks down tasks, assigns them to agents, and coordinates results.

How Agent Teams Work

Each agent has an **independent context window**, its own **worktree**, and an **assigned scope**. Communication happens through the team lead, not directly between agents.

Good fit: Large features with separable sub-tasks, independent modules

Poor fit: Tightly coupled code, small tasks where coordination overhead exceeds benefit

Remember: This is experimental. Expect rough edges. Use worktrees and sub-agents for production workflows today.

Code Review for AI-Generated Code

> *Reviewing AI code requires different skills than reviewing human code*

Why AI Code Review Is Different

AI-generated code has **specific failure patterns** that human code doesn't:

Research findings (ClackyAI study):

- **1.57x** more security issues than human-written code
- **2.74x** more cross-site scripting (XSS) vulnerabilities
- **1.91x** more insecure direct object references

The code *looks* correct. It passes a quick scan. But it hides subtle issues.

AI-Specific Pitfalls

False confidence: AI code is well-formatted and commented, creating an illusion of quality. Reviewers let their guard down.

Hallucinated APIs: The AI calls functions or methods that don't exist. Simon Willison notes these are actually "the least dangerous" kind of hallucination -- they cause immediate, visible errors.

Duplicate logic: Without global project awareness, AI generates redundant implementations. One team found **11 different email validation implementations** in their AI-assisted codebase.

Stale patterns: AI may use deprecated APIs or outdated patterns from its training data.

The C.L.E.A.R. Framework

A structured approach to reviewing AI-generated code:

C -- Context: Does this code fit the project's architecture and conventions?

L -- Logic: Is the business logic correct? Are edge cases handled?

E -- Evidence: Are there tests? Do they actually verify the behavior?

A -- Architecture: Does it follow established patterns? Any new dependencies?

R -- Risk: Security issues? Performance concerns? Data exposure?

C.L.E.A.R. in Practice

Step	What to Check	AI-Specific Risk
Context	Fits conventions? Uses right patterns?	AI may invent its own conventions
Logic	Edge cases? Error handling?	AI often handles happy path only
Evidence	Tests exist? Tests are meaningful?	AI writes tests that pass but don't verify behavior
Architecture	Follows patterns? No new deps?	AI adds unnecessary libraries
Risk	SQL injection? XSS? Auth checks?	1.57x more security issues

The Writer/Reviewer Pattern

Use Claude Code itself as part of the review process:

1. **WRITE** -- Agent 1 implements the feature
2. **REVIEW** -- Agent 2 (sub-agent) reviews using C.L.E.A.R.
3. **REVISE** -- Agent 1 addresses findings
4. **HUMAN REVIEW** -- You review the final result

Never skip Step 4. AI reviewing AI is useful but not sufficient. A human must sign off.

PR Metadata for AI Code

Add transparency to your pull requests:

```
### AI Disclosure
- **% AI-generated:** ~80%
- **AI tool:** Claude Code
- **Human review:** Yes, C.L.E.A.R. applied
- **Security check:** Input validation verified,
  no raw SQL, auth middleware confirmed
```

Including a "% AI-generated" field helps reviewers know where to focus their attention.

Putting It All Together

```
.claude/
  skills/fix-issue.md       Standardized workflows
  agents/security-reviewer.md   Specialized reviewers
  settings.json
Hooks for formatting/linting
.mcp.json
     Shared MCP server config
CLAUDE.md
     Project context & conventions
```

The full workflow:

1. Pick issue from scrumboard
2. `claude --worktree` then `/fix-issue #42`
3. Hooks auto-format on every edit
4. MCP servers provide DB schema, test data
5. Sub-agent reviews for security
6. `/create-pr` -- teammate reviews with C.L.E.A.R.
7. Merge and move issue to Done

Hands-on Lab

> *Build your extensibility toolkit*

Exercise 1: Build a Custom Skill (15 min)

Create a skill in `.claude/skills/` for your P3 project (e.g., `add-api-route.md`). Include a description, step-by-step instructions, project conventions, and constraints. Test it by typing the slash command in Claude Code.

Exercise 2: Configure Hooks (10 min)

Add a PostToolUse hook that runs Prettier on `Edit|Write`, and a PreToolUse hook that blocks edits to `.env` files (exit code 2).
Configure in `.claude/settings.json`. Verify by editing a file and confirming Prettier runs automatically.

Exercise 3: Connect an MCP Server (10 min)

Add an MCP server to your project (Playwright, filesystem, or database). Use `claude mcp add` and verify by asking Claude Code to use the connected service. Add the config to `.mcp.json` for your team.

Exercise 4: Parallel Development Sprint (15 min)

Split your team and run parallel agents:

```
Teammate A: claude --worktree
  > "Implement [feature from scrumboard]"

Teammate B: claude --worktree
  > "Implement [different feature]"

Teammate C: claude --worktree
  > "Write tests for [existing module]"
```

After 15 minutes: review, resolve conflicts, merge.

This Week's Deliverables

Due This Week

HW5: Custom Skill + MCP Integration

- Create a custom skill for your P3 project
- Connect at least one MCP server
- Configure at least one hook
- Document your extensibility setup in CLAUDE.md

Weekly Quiz 12 -- Claude Code Extensibility concepts

P3 Sprint 2 -- continue sprint work with your team

Next Week Preview

Week 13: Agent Architectures & Agent SDK

- **Agent fundamentals** -- agent vs workflow, augmented LLM
- **Anthropic's 6 agent patterns** -- chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, autonomous
- **Claude Agent SDK** -- Python/TypeScript, query(), hooks, sessions
- **Multi-agent coordination** and safety
- **Real-world agent examples**

Resources

Required Reading

Resource	URL
Claude Code in Action (Modules 5-6)	**anthropic.skilljar.com/claude-code-in-action**
Skills Documentation	**code.claude.com/docs/en/skills**
Hooks Documentation	**code.claude.com/docs/en/hooks**
MCP Servers Documentation	**code.claude.com/docs/en/mcp**
Sub-agents Documentation	**code.claude.com/docs/en/sub-agents**

Recommended Reading

Resource	URL
MCP Specification	**modelcontextprotocol.io**
MCP GitHub Repository	**github.com/modelcontextprotocol**
Playwright MCP Server	**github.com/anthropics/mcp-servers/tree/main/playwright**
Claude Code Plugins	**code.claude.com/docs/en/plugins**

Speaker notes